

Série d'Activités Pédagogiques :

Paradigmes Algorithmiques - ÉNONCÉS

Travaux Dirigés et Travaux Pratiques

Table des matières

1	Introduction Pédagogique	1
1.1	Objectifs des Activités	1
1.2	Structure des Séances	2
2	Activité 1 : Classification et Complexité	2
2.1	Objectifs Spécifiques	2
3	Activité 2 : Diviser pour Régner	4
3.1	Objectifs Spécifiques	4
4	Activité 3 : Programmation Dynamique	7
4.1	Objectifs Spécifiques	7
5	Activité 4 : Algorithmes Gloutons	10
5.1	Objectifs Spécifiques	10
6	Activité 5 : Backtracking et Branch and Bound	13
6.1	Objectifs Spécifiques	13
7	Activité 6 : Recherche Locale et Méta-heuristiques	18
7.1	Objectifs Spécifiques	18

1 Introduction Pédagogique

1.1 Objectifs des Activités

Cette série d'activités vise à développer chez les étudiants :

- Une compréhension profonde des paradigmes algorithmiques
- La capacité à analyser et choisir le paradigme approprié
- Les compétences d'implémentation et d'optimisation
- L'analyse critique des performances algorithmiques
- La résolution créative de problèmes complexes

1.2 Structure des Séances

Chaque séance comprend :

1. **Partie théorique** : Rappels et approfondissements
2. **Travaux dirigés** : Exercices guidés d'analyse
3. **Travaux pratiques** : Implémentation et expérimentation
4. **Synthèse** : Discussion et retours d'expérience

2 Activité 1 : Classification et Complexité

2.1 Objectifs Spécifiques

- Comprendre la hiérarchie des classes de complexité
- Analyser la complexité d'algorithmes simples
- Distinguer problèmes faciles et difficiles

Travail Dirigé 1.1 : Analyse de Complexité

Exercices :

1. Pour chaque algorithme suivant, déterminer la complexité asymptotique :

```
# Algorithme 1
for i in range(n):
  for j in range(n):
    print(i, j)
```

```
# Algorithme 2
for i in range(n):
  for j in range(i):
    print(i, j)
```

```
# Algorithme 3
i = n
while i > 0:
  print(i)
  i = i // 2
```

```
# Algorithme 4
for i in range(n):
  for j in range(m):
    for k in range(m):
      print(i, j, k)
```

2. Classer les problèmes suivants selon P, NP, NP-complet :
 - Trouver le plus court chemin dans un graphe
 - Résoudre un Sudoku 9×9
 - Vérifier si un graphe est biparti
 - Problème du voyageur de commerce

Travail Pratique 1.2 : Expérimentation Empirique

Objectif : Mesurer les temps d'exécution réels vs théoriques.

Implémentation :

```
import time
import matplotlib.pyplot as plt
import numpy as np

def algorithme_quadratique(n):
    #  $O(n^2)$ 
    somme = 0
    for i in range(n):
        for j in range(n):
            somme += i * j
    return somme

def algorithme_lineaire(n):
    #  $O(n)$ 
    somme = 0
    for i in range(n):
        somme += i
    return somme

def mesurer_temps():
    tailles = [10, 50, 100, 500, 1000, 2000]
    temps_quad = []
    temps_lin = []

    for n in tailles:
        # Mesure temps quadratique
        start = time.time()
        algorithme_quadratique(n)
        temps_quad.append(time.time() - start)

        # Mesure temps lin aire
        start = time.time()
        algorithme_lineaire(n)
        temps_lin.append(time.time() - start)

    # Affichage des r sultats
    plt.figure(figsize=(10, 6))
    plt.plot(tailles, temps_quad, 'r-', label='O( $n^2$ )')
    plt.plot(tailles, temps_lin, 'b-', label='O( $n$ )')
    plt.xlabel('Taille_de_l\'entr e_(n)')
    plt.ylabel('Temps_d\'ex cution_(s)')
    plt.title('Croissance_des_temps_d\'ex cution')
    plt.legend()
```

```
plt.grid(True)
plt.savefig('complexite_empirique.png')
plt.show()

# Analyse du rapport
print("Rapport_des_temps_(quadratique/lin_aire):")
for i in range(len(tailles)):
    rapport = temps_quad[i] / temps_lin[i] if temps_lin[i] > 0 else 0
    print(f"n={tailles[i]}: {rapport:.2f}")
```

Questions :

1. Tracer les courbes de croissance et les comparer aux prédictions théoriques
2. Pourquoi observe-t-on des écarts par rapport aux prédictions théoriques ?
3. Quelle serait la taille maximale de problème pour un algorithme $O(n^3)$ sur votre machine ?

3 Activité 2 : Diviser pour Régner

3.1 Objectifs Spécifiques

- Maîtriser la récursivité et la décomposition
- Analyser les équations de récurrence
- Implémenter des algorithmes classiques

Travail Dirigé 2.1 : Résolution d'Équations de Récurrence

Exercice 2.1.1 : Master Theorem

Résoudre les équations suivantes en utilisant le Master Theorem :

1. $T(n) = 2T(n/2) + n$
2. $T(n) = 4T(n/2) + n^2$
3. $T(n) = 8T(n/2) + n^3$
4. $T(n) = 7T(n/2) + n^2$
5. $T(n) = T(n/2) + \log n$

Exercice 2.1.2 : Analyse d'Algorithme

Considérons l'algorithme suivant qui trouve le maximum et le minimum dans un tableau :

Algorithm 1 MaxMin Diviser pour Régner

```
1: function MAXMIN( $A, l, r$ )
2:   if  $l = r$  then
3:     return ( $A[l], A[l]$ )
4:   else if  $r = l + 1$  then
5:     if  $A[l] < A[r]$  then
6:       return ( $A[l], A[r]$ )
7:     else
8:       return ( $A[r], A[l]$ )
9:     end if
10:  else
11:     $m \leftarrow \lfloor (l + r) / 2 \rfloor$ 
12:    ( $min1, max1$ )  $\leftarrow$  MAXMIN( $A, l, m$ )
13:    ( $min2, max2$ )  $\leftarrow$  MAXMIN( $A, m + 1, r$ )
14:    return ( $\min(min1, min2), \max(max1, max2)$ )
15:  end if
16: end function
```

1. Écrire l'équation de récurrence
2. Résoudre cette équation
3. Comparer avec l'algorithme itératif simple

Travail Pratique 2.2 : Implémentation et Comparaison

Objectif : Implémenter et comparer différentes versions d'un même problème.

Listing 1 – Multiplication de matrices

```
import numpy as np
import time

def multiplication_naive(A, B):
    """Multiplication_naive_O( $n^3$ )"""
    n = len(A)
    C = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def multiplication_strassen(A, B):
```

```

"""Algorithme_de_Strassen_O(n^log2(7))"""
n = len(A)

# Cas de base
if n == 1:
    return [[A[0][0] * B[0][0]]]

# D coupage des matrices
m = n // 2
A11 = [row[:m] for row in A[:m]]
A12 = [row[m:] for row in A[:m]]
A21 = [row[:m] for row in A[m:]]
A22 = [row[m:] for row in A[m:]]

B11 = [row[:m] for row in B[:m]]
B12 = [row[m:] for row in B[:m]]
B21 = [row[:m] for row in B[m:]]
B22 = [row[m:] for row in B[m:]]

# Calcul des 7 produits de Strassen
M1 = multiplication_strassen(
    add_matrices(A11, A22),
    add_matrices(B11, B22)
)
M2 = multiplication_strassen(
    add_matrices(A21, A22),
    B11
)
# ... (autres produits)

# Reconstruction
C11 = add_matrices(sub_matrices(add_matrices(M1, M4), M5), M7)
# ... (autres blocs)

# Combinaison
C = []
for i in range(m):
    C.append(C11[i] + C12[i])
for i in range(m):
    C.append(C21[i] + C22[i])

return C

def comparer_performances():
    """Compare les performances des deux algorithmes"""
    tailles = [2, 4, 8, 16, 32, 64]
    temps_naif = []

```

```

temps_strassen = []

for n in tailles:
    A = np.random.rand(n, n)
    B = np.random.rand(n, n)

    # Multiplication naive
    start = time.time()
    multiplication_naive(A, B)
    temps_naif.append(time.time() - start)

    # Strassen (seulement pour n pair et suffisamment grand)
    if n >= 8:
        start = time.time()
        multiplication_strassen(A, B)
        temps_strassen.append(time.time() - start)
    else:
        temps_strassen.append(None)

# Analyse des r sultats
print("Temps_d'ex cution_(s):")
print("n\tNaive\tStrassen\tRapport")
for i, n in enumerate(tailles):
    rapport = (temps_naif[i] / temps_strassen[i]
    if temps_strassen[i] else None)
    print(f"{n}\t{temps_naif[i]:.4 f}\t"
    f"{temps_strassen[i]_or_'N/A':.4 f}\t"
    f"{rapport_or_'N/A':.2 f}")

```

Tâches :

1. Compléter l'implémentation de Strassen
2. Mesurer les performances pour différentes tailles
3. Identifier le seuil à partir duquel Strassen devient plus efficace
4. Analyser l'impact de la constante cachée dans la complexité

4 Activité 3 : Programmation Dynamique

4.1 Objectifs Spécifiques

- Identifier les problèmes à sous-structure optimale
- Concevoir des relations de récurrence
- Implémenter les versions bottom-up et top-down

Travail Dirigé 3.1 : Conception de Relations de Récurrence

Exercice 3.1.1 : Sous-séquence croissante la plus longue

Pour un tableau $A[1..n]$, on cherche la longueur de la plus longue sous-séquence croissante.

1. Définir $L[i]$ = longueur de la LIS se terminant à la position i
2. Écrire la relation de récurrence
3. Initialiser correctement les valeurs de base
4. Proposer un algorithme pour reconstruire la solution

Exercice 3.1.2 : Problème du rendu de monnaie

Étant donné un système de pièces $c_1, c_2, \dots, c_m$ et un montant S , trouver le nombre minimum de pièces pour obtenir S .

1. Définir $dp[x]$ = nombre minimum de pièces pour obtenir le montant x
2. Écrire la relation de récurrence
3. Discuter de l'optimalité pour différents systèmes de pièces
4. Adapter pour compter le nombre de façons de rendre la monnaie

Travail Pratique 3.2 : Optimisation Mémoire

Objectif : Implémenter et optimiser l'utilisation mémoire en DP.

Listing 2 – Optimisation mémoire pour le sac à dos

```
def knapsack_2D(W, wt, val, n):
    """Version_2D_classique"""
    K = [[0 for _ in range(W+1)] for _ in range(n+1)]

    for i in range(1, n+1):
        for w in range(1, W+1):
            if wt[i-1] <= w:
                K[i][w] = max(val[i-1] + K[i-1][w-wt[i-1]],
                               K[i-1][w])
            else:
                K[i][w] = K[i-1][w]

    return K[n][W]

def knapsack_1D(W, wt, val, n):
    """Version_1D_optimisee_en_memoire"""
    dp = [0 for _ in range(W+1)]

    for i in range(1, n+1):
        # Parcours inverse pour éviter la r utilisation
```



```

for w in range(W, wt[i-1]-1, -1):
    dp[w] = max(dp[w],
    val[i-1] + dp[w - wt[i-1]])

return dp[W]

def knapsack_approximation(W, wt, val, n, epsilon=0.1):
    """Version d'approximation FPTAS"""
    # chelle des valeurs
    max_val = max(val)
    K = epsilon * max_val / n
    scaled_val = [int(v/K) for v in val]

    # DP sur valeurs reduites
    total_val = sum(scaled_val)
    dp = [[float('inf') for _ in range(total_val+1)]
    for _ in range(n+1)]

    dp[0][0] = 0
    for i in range(1, n+1):
        for v in range(total_val+1):
            if scaled_val[i-1] <= v:
                dp[i][v] = min(dp[i-1][v],
                dp[i-1][v-scaled_val[i-1]] + wt[i-1])
            else:
                dp[i][v] = dp[i-1][v]

    # Trouver la meilleure valeur realisable
    best_val = 0
    for v in range(total_val+1):
        if dp[n][v] <= W and v > best_val:
            best_val = v

    return best_val * K # Valeur approximee

```

Expérimentation :

1. Comparer l'utilisation mémoire des versions 2D et 1D
2. Mesurer l'impact sur le temps d'exécution
3. Tester l'approximation FPTAS avec différentes valeurs d'epsilon
4. Analyser le compromis précision/temps

Questions :

1. Pourquoi le parcours inverse dans la version 1D ?
2. Quel est le ratio d'approximation garanti par FPTAS ?
3. Dans quels cas préférer l'approximation à la solution exacte ?

5 Activité 4 : Algorithmes Gloutons

5.1 Objectifs Spécifiques

- Identifier la propriété du choix glouton
- Prouver l'optimalité des algorithmes gloutons
- Analyser les cas où les gloutons échouent

Travail Dirigé 4.1 : Preuves d'Optimalité

Exercice 4.1.1 : Ordonnement d'activités

On a n activités avec heures de début s_i et fin f_i . Trouver le nombre maximum d'activités compatibles.

1. Proposer une stratégie gloutonne
2. Prouver son optimalité par échange
3. Analyser la complexité
4. Généraliser avec des poids sur les activités

Exercice 4.1.2 : Contre-exemple du glouton

Pour le système de pièces $\{1, 3, 4\}$ et le montant 6, montrer que la stratégie gloutonne (prendre la plus grande pièce possible) n'est pas optimale.

1. Appliquer la stratégie gloutonne
2. Trouver une solution meilleure
3. Caractériser les systèmes de pièces où le glouton fonctionne

Travail Pratique 4.2 : Benchmark des Algorithmes d'Ordonnement

Objectif : Implémenter et comparer différents algorithmes d'ordonnement.

Listing 3 – Algorithmes d'ordonnement

```
import random
from dataclasses import dataclass
from typing import List
import matplotlib.pyplot as plt

@dataclass
class Job:
    id: int
    processing_time: int
    due_date: int
    weight: float = 1.0

def earliest_due_date(jobs: List[Job]) -> List[Job]:
    """ Rgle EDD (Earliest Due Date) """
```

```

return sorted(jobs , key=lambda j: j.due_date)

def weighted_shortest_processing_time(jobs: List[Job]) -> List[Job]:
    """Règle WSPT (Weighted Shortest Processing Time)"""
    return sorted(jobs , key=lambda j: j.processing_time/j.weight)

def moore_hodgson(jobs: List[Job]) -> List[Job]:
    """Algorithme de Moore-Hodgson pour minimiser le nombre de retards"""
    # Trier par date d'arrivée
    sorted_jobs = sorted(jobs , key=lambda j: j.due_date)
    schedule = []
    rejected = []
    current_time = 0

    for job in sorted_jobs:
        # Ajouter le job à la schedule
        schedule.append(job)
        current_time += job.processing_time

        # Vérifier si le job est en retard
        if current_time > job.due_date:
            # Retirer le job avec le plus grand temps de traitement
            max_processing = max(schedule , key=lambda j: j.processing_time)
            schedule.remove(max_processing)
            rejected.append(max_processing)
            current_time -= max_processing.processing_time

    return schedule , rejected

def evaluate_schedule(schedule: List[Job] , metric: str = "tardiness") -> float:
    """Calculer la valeur d'une schedule selon différentes métriques"""
    current_time = 0
    total = 0

    if metric == "tardiness":
        for job in schedule:
            current_time += job.processing_time
            tardiness = max(0 , current_time - job.due_date)
            total += tardiness * job.weight

    elif metric == "makespan":
        total = sum(job.processing_time for job in schedule)

    elif metric == "lateness":
        current_time = 0
        for job in schedule:
            current_time += job.processing_time

```

```

lateness = current_time - job.due_date
total += lateness * job.weight

return total

def generate_random_jobs(n: int) -> List[Job]:
    """G n re_n_jobs_ al_atoires"""
    jobs = []
    for i in range(n):
        p = random.randint(1, 20) # Temps de traitement
        d = random.randint(p, 5*p) # Date d' chance
        w = random.uniform(0.5, 2.0) # Poids
        jobs.append(Job(i, p, d, w))
    return jobs

def benchmark_algorithms():
    """Compare les performances des diff rents algorithmes"""
    n_jobs_list = [10, 20, 50, 100, 200]
    metrics = ["tardiness", "makespan", "lateness"]
    results = {metric: {n: {} for n in n_jobs_list} for metric in metrics}

    for n in n_jobs_list:
        for _ in range(10): # 10 instances par taille
            jobs = generate_random_jobs(n)

            # Appliquer les algorithmes
            edd_schedule = earliest_due_date(jobs.copy())
            wspt_schedule = weighted_shortest_processing_time(jobs.copy())
            mh_schedule, _ = moore_hodgson(jobs.copy())

            # valuer les m triques
            for metric in metrics:
                results[metric][n].setdefault("EDD", []).append(
                    evaluate_schedule(edd_schedule, metric))
                results[metric][n].setdefault("WSPT", []).append(
                    evaluate_schedule(wspt_schedule, metric))
                results[metric][n].setdefault("Moore-Hodgson", []).append(
                    evaluate_schedule(mh_schedule, metric))

            # Analyser les r sultats
            for metric in metrics:
                plt.figure(figsize=(10, 6))
                for algo in ["EDD", "WSPT", "Moore-Hodgson"]:
                    means = [np.mean(results[metric][n][algo]) for n in n_jobs_list]
                    plt.plot(n_jobs_list, means, 'o-', label=algo)

            plt.xlabel('Nombre_de_jobs')

```

```
plt.ylabel(f'Métrique : {metric}')
plt.title(f'Performance des algorithmes - {metric}')
plt.legend()
plt.grid(True)
plt.savefig(f'benchmark_{metric}.png')
plt.show()
```

Tâches :

1. Implémenter les algorithmes manquants
2. Générer des benchmarks avec différentes distributions de jobs
3. Analyser les performances selon différentes métriques
4. Identifier les cas où chaque algorithme excelle

6 Activité 5 : Backtracking et Branch and Bound

6.1 Objectifs Spécifiques

- Maîtriser l'exploration systématique d'espaces de solutions
- Concevoir des stratégies d'élagage efficaces
- Comparer backtracking et branch and bound

Travail Dirigé 5.1 : Conception d'Élagages

Exercice 5.1.1 : Problème de la somme de sous-ensemble

Étant donné un ensemble d'entiers positifs S et une cible T , trouver un sous-ensemble dont la somme est exactement T .

1. Proposer un algorithme de backtracking naïf
2. Ajouter des élagages par faisabilité
3. Ajouter des élagages par optimalité
4. Proposer une heuristique d'ordre de traitement

Exercice 5.1.2 : Problème d'affectation de tâches

n tâches à affecter à n machines, chaque machine i a un coût c_{ij} pour la tâche j . Trouver l'affectation de coût minimum.

1. Formuler comme problème de permutation
2. Proposer une borne inférieure pour le branch and bound
3. Comparer avec l'algorithme hongrois (complexité polynomiale)

Travail Pratique 5.2 : Solveur de Sudoku

Objectif : Implémenter un solveur de Sudoku avec différentes stratégies.

Listing 4 – Solveur de Sudoku

```
class SudokuSolver:
    def __init__(self, grid):
        self.grid = grid
        self.n = len(grid)
        self.sub_n = int(self.n ** 0.5)
        self.solutions = []

    def is_valid(self, row, col, num):
        """Vérifie si num peut être placé à (row, col)"""
        # Ligne
        for x in range(self.n):
            if self.grid[row][x] == num:
                return False

        # Colonne
        for x in range(self.n):
            if self.grid[x][col] == num:
                return False

        # Sous-carre
        start_row = row - row % self.sub_n
        start_col = col - col % self.sub_n
        for i in range(self.sub_n):
            for j in range(self.sub_n):
                if self.grid[i + start_row][j + start_col] == num:
                    return False

        return True

    def find_empty_location(self):
        """Trouve une case vide avec MRV (Minimum Remaining Values)"""
        min_options = self.n + 1
        best_pos = None

        for i in range(self.n):
            for j in range(self.n):
                if self.grid[i][j] == 0:
                    # Compter les options possibles
                    options = 0
                    for num in range(1, self.n + 1):
                        if self.is_valid(i, j, num):
                            options += 1
```

```

if options < min_options:
    min_options = options
    best_pos = (i, j)

return best_pos

def forward_checking(self, row, col, num):
    """Vérification prospective"""
    # Créer une copie des domaines
    domains = [[set(range(1, self.n + 1))
    if self.grid[i][j] == 0 else {self.grid[i][j]}
    for j in range(self.n)] for i in range(self.n)]

    # Mettre à jour les domaines
    for i in range(self.n):
        if num in domains[row][i]:
            domains[row][i].remove(num)
            if len(domains[row][i]) == 0:
                return False

    for i in range(self.n):
        if num in domains[i][col]:
            domains[i][col].remove(num)
            if len(domains[i][col]) == 0:
                return False

    # Sous-carre
    start_row = row - row % self.sub_n
    start_col = col - col % self.sub_n
    for i in range(self.sub_n):
        for j in range(self.sub_n):
            r, c = start_row + i, start_col + j
            if num in domains[r][c]:
                domains[r][c].remove(num)
                if len(domains[r][c]) == 0:
                    return False

    return True

def solve_backtracking(self, count_all=False):
    """Backtracking simple"""
    pos = self.find_empty_location()

    if not pos: # Plus de cases vides
        self.solutions.append([row[:] for row in self.grid])
    return len(self.solutions) > 0

```

```

row, col = pos

for num in range(1, self.n + 1):
    if self.is_valid(row, col, num):
        self.grid[row][col] = num

    if self.solve_backtracking(count_all):
        if not count_all: # Une solution suffit
        return True

self.grid[row][col] = 0 # Backtrack

return False

def solve_with_forward_checking(self, count_all=False):
    """Backtracking_avec_v rification_prospective"""
    pos = self.find_empty_location()

    if not pos:
        self.solutions.append([row[:] for row in self.grid])
        return len(self.solutions) > 0

    row, col = pos

    # Trier les nombres par nombre de contraintes (Least Constraining Value)
    numbers = list(range(1, self.n + 1))
    numbers.sort(key=lambda num: self.count_constraints(row, col, num))

    for num in numbers:
        if self.is_valid(row, col, num) and self.forward_checking(row, col, num):
            self.grid[row][col] = num

    if self.solve_with_forward_checking(count_all):
        if not count_all:
            return True

    self.grid[row][col] = 0

    return False

def count_constraints(self, row, col, num):
    """Compte combien de cases voisines sont contraintes par num"""
    constraints = 0

    # Cases sur la m me ligne/colonne
    for i in range(self.n):
        if self.grid[row][i] == 0 and i != col:

```



```

if not self.is_valid(row, i, num):
    constraints += 1

for i in range(self.n):
    if self.grid[i][col] == 0 and i != row:
    if not self.is_valid(i, col, num):
        constraints += 1

# Cases dans le m me sous-carr
start_row = row - row % self.sub_n
start_col = col - col % self.sub_n
for i in range(self.sub_n):
for j in range(self.sub_n):
    r, c = start_row + i, start_col + j
    if self.grid[r][c] == 0 and (r, c) != (row, col):
    if not self.is_valid(r, c, num):
        constraints += 1

return constraints

def benchmark_solvers(self, puzzles):
    """Compare les performances des diff rentes m thodes"""
    results = []

    for puzzle in puzzles:
        # Backtracking simple
        solver1 = SudokuSolver([row[:] for row in puzzle])
        start = time.time()
        solver1.solve_backtracking()
        time1 = time.time() - start

        # Avec forward checking
        solver2 = SudokuSolver([row[:] for row in puzzle])
        start = time.time()
        solver2.solve_with_forward_checking()
        time2 = time.time() - start

        results.append({
            'size': len(puzzle),
            'backtracking_time': time1,
            'forward_checking_time': time2,
            'speedup': time1 / time2 if time2 > 0 else float('inf')
        })

    return results

Expérimentation :

```

1. Générer des grilles de Sudoku de différentes difficultés
2. Comparer les temps de résolution avec différentes stratégies
3. Mesurer l'impact de MRV et LCV
4. Tester sur des grilles 4×4 , 9×9 , 16×16

7 Activité 6 : Recherche Locale et Méta-heuristiques

7.1 Objectifs Spécifiques

- Implémenter des méta-heuristiques classiques
- Paramétrer et optimiser les algorithmes
- Analyser la convergence et la qualité des solutions

Travail Dirigé 6.1 : Analyse de Convergence

Exercice 6.1.1 : Recuit simulé

Pour le recuit simulé appliqué au TSP :

1. Définir une fonction de voisinage (2-opt swap)
2. Proposer une fonction de décroissance de température
3. Analyser l'impact des paramètres sur la convergence
4. Comparer avec une descente de gradient simple

Exercice 6.1.2 : Algorithmes génétiques

Pour un algorithme génétique résolvant le problème du sac à dos :

1. Définir une représentation des chromosomes
2. Proposer des opérateurs de croisement et mutation
3. Analyser l'impact de la taille de population
4. Évaluer la diversité génétique au cours des générations

Travail Pratique 6.2 : Framework de Méta-heuristiques

Objectif : Implémenter un framework modulaire pour comparer différentes méta-heuristiques.

Listing 5 – Framework de méta-heuristiques

```
from abc import ABC, abstractmethod
import random
import numpy as np
from typing import List, Tuple, Callable, Any
import matplotlib.pyplot as plt
```

```

class Problem(ABC):
    """Classe abstraite pour un problème d'optimisation"""

    @abstractmethod
    def evaluate(self, solution: Any) -> float:
        """value d'une solution"""
    pass

    @abstractmethod
    def generate_random_solution(self) -> Any:
        """Génère une solution aléatoire"""
    pass

    @abstractmethod
    def get_neighborhood(self, solution: Any) -> List[Any]:
        """Retourne le voisinage d'une solution"""
    pass

class TSPProblem(Problem):
    """Implémentation pour le TSP"""

    def __init__(self, distances: np.ndarray):
        self.distances = distances
        self.n = len(distances)

    def evaluate(self, tour: List[int]) -> float:
        """Calcule la longueur du tour"""
        total = 0
        for i in range(self.n):
            total += self.distances[tour[i]][tour[(i+1) % self.n]]
        return total

    def generate_random_solution(self) -> List[int]:
        """Génère une permutation aléatoire"""
        tour = list(range(self.n))
        random.shuffle(tour)
        return tour

    def get_neighborhood(self, tour: List[int]) -> List[List[int]]:
        """Génère le voisinage par swaps 2-opt"""
        neighbors = []
        n = len(tour)

        for i in range(n):
            for j in range(i+2, n):
                if j == n-1 and i == 0:
                    continue # éviter la swap avec le dernier/first

```

```

# Cr er un nouveau tour par inversion de segment
new_tour = tour [:]
if i < j:
    new_tour[i:j+1] = reversed(new_tour[i:j+1])
else:
    segment = new_tour[i:] + new_tour[:j+1]
    segment.reverse()
    new_tour[i:] = segment[:n-i]
    new_tour[:j+1] = segment[n-i:]

neighbors.append(new_tour)

return neighbors

class Metaheuristic(ABC):
    """Classe abstraite pour une meta-heuristique"""

    def __init__(self, problem: Problem, max_iter: int = 1000):
        self.problem = problem
        self.max_iter = max_iter
        self.best_solution = None
        self.best_value = float('inf')
        self.history = []

    @abstractmethod
    def solve(self) -> Tuple[Any, float]:
        """R soudre le probl me"""
        pass

    def plot_convergence(self, title: str = ""):
        """Affiche la courbe de convergence"""
        plt.figure(figsize=(10, 6))
        plt.plot(self.history, 'b-', linewidth=2)
        plt.xlabel('It ration')
        plt.ylabel('Meilleure valeur')
        plt.title(f'Convergence_{title}')
        plt.grid(True, alpha=0.3)
        plt.show()

class SimulatedAnnealing(Metaheuristic):
    """Impl mentation du recuit simul e"""

    def __init__(self, problem: Problem, max_iter: int = 1000,
temp_init: float = 1000, cooling_rate: float = 0.995):
        super().__init__(problem, max_iter)
        self.temp_init = temp_init

```

```

self.cooling_rate = cooling_rate

def solve(self) -> Tuple[Any, float]:
    current = self.problem.generate_random_solution()
    current_value = self.problem.evaluate(current)
    self.best_solution = current[:]
    self.best_value = current_value

    temperature = self.temp_init

    for iteration in range(self.max_iter):
        # Choisir un voisin alatoire
        neighbors = self.problem.get_neighborhood(current)
        if not neighbors:
            break

        neighbor = random.choice(neighbors)
        neighbor_value = self.problem.evaluate(neighbor)

        # Crit re d'acceptation
        delta = neighbor_value - current_value

        if delta < 0 or random.random() < np.exp(-delta / temperature):
            current = neighbor
            current_value = neighbor_value

        if current_value < self.best_value:
            self.best_solution = current[:]
            self.best_value = current_value

        # Refroidissement
        temperature *= self.cooling_rate
        self.history.append(self.best_value)

    return self.best_solution, self.best_value

class GeneticAlgorithm(Metaheuristic):
    """Impl mentation d'algorithme g n tique"""

    def __init__(self, problem: Problem, max_iter: int = 1000,
                  pop_size: int = 50, crossover_rate: float = 0.8,
                  mutation_rate: float = 0.2, tournament_size: int = 3):
        super().__init__(problem, max_iter)
        self.pop_size = pop_size
        self.crossover_rate = crossover_rate
        self.mutation_rate = mutation_rate
        self.tournament_size = tournament_size

```

```

def initialize_population(self):
    """Initialise la population"""
    return [self.problem.generate_random_solution()
            for _ in range(self.pop_size)]

def tournament_selection(self, population: List[Any],
                          values: List[float]) -> Any:
    """Sélection par tournoi"""
    tournament = random.sample(list(zip(population, values)),
                                self.tournament_size)
    return min(tournament, key=lambda x: x[1])[0]

def order_crossover(self, parent1: List[int], parent2: List[int]) -> List[int]:
    """Croisement par ordre pour TSP"""
    n = len(parent1)
    i, j = sorted(random.sample(range(n), 2))

    child = [-1] * n
    child[i:j+1] = parent1[i:j+1]

    # Remplir le reste avec les éléments de parent2
    pos = (j + 1) % n
    for k in range(n):
        elem = parent2[(j + 1 + k) % n]
        if elem not in child:
            child[pos] = elem
            pos = (pos + 1) % n

    return child

def swap_mutation(self, individual: List[int]) -> List[int]:
    """Mutation par swap"""
    if random.random() < self.mutation_rate:
        i, j = random.sample(range(len(individual)), 2)
        individual[i], individual[j] = individual[j], individual[i]
    return individual

def solve(self) -> Tuple[Any, float]:
    population = self.initialize_population()
    values = [self.problem.evaluate(ind) for ind in population]

    # Initialiser la meilleure solution
    best_idx = np.argmin(values)
    self.best_solution = population[best_idx][:]
    self.best_value = values[best_idx]

```

```

for generation in range(self.max_iter):
    new_population = []

    # litisme : conserver la meilleure solution
    new_population.append(self.best_solution)

    while len(new_population) < self.pop_size:
        # S lection
        parent1 = self.tournament_selection(population, values)
        parent2 = self.tournament_selection(population, values)

        # Croisement
        if random.random() < self.crossover_rate:
            child = self.order_crossover(parent1, parent2)
        else:
            child = random.choice([parent1, parent2])

        # Mutation
        child = self.swap_mutation(child)

        new_population.append(child)

    # Mise jour
    population = new_population
    values = [self.problem.evaluate(ind) for ind in population]

    # Mettre jour la meilleure solution
    current_best_idx = np.argmin(values)
    if values[current_best_idx] < self.best_value:
        self.best_solution = population[current_best_idx][:]
        self.best_value = values[current_best_idx]

    self.history.append(self.best_value)

return self.best_solution, self.best_value

def benchmark_metaheuristics(problem: Problem, n_runs: int = 10):
    """Compare différentes méta-heuristiques"""
    results = {
        'Simulated_Annealing': [],
        'Genetic_Algorithm': [],
        'Tabu_Search': []
    }

    for _ in range(n_runs):
        # Recuit simulé
        sa = SimulatedAnnealing(problem, max_iter=1000)

```

```

sa_solution, sa_value = sa.solve()
results['Simulated_Annealing'].append(sa_value)

# Algorithme génétique
ga = GeneticAlgorithm(problem, max_iter=100)
ga_solution, ga_value = ga.solve()
results['Genetic_Algorithm'].append(ga_value)

# Analyse statistique
print("Résultats (moyenne et écart-type):")
for method, values in results.items():
    mean_val = np.mean(values)
    std_val = np.std(values)
    print(f"{method}: {mean_val:.2f} ± {std_val:.2f}")

# Visualisation
plt.figure(figsize=(12, 8))

# Box plot des résultats finaux
plt.subplot(2, 2, 1)
plt.boxplot(results.values())
plt.xticks(range(1, len(results)+1), results.keys())
plt.ylabel('Valeur de la solution')
plt.title('Distribution des résultats')

# Courbes de convergence moyennes
plt.subplot(2, 2, 2)
# (implémenter avec stockage des historiques)

# Temps de calcul
plt.subplot(2, 2, 3)
# (implémenter avec mesure du temps)

plt.tight_layout()
plt.show()

```

Expérimentation :

1. Comparer les performances sur différentes instances de TSP
2. Étudier l'impact des paramètres sur la qualité des solutions
3. Mesurer les temps de convergence
4. Analyser la robustesse (écart-type sur plusieurs runs)